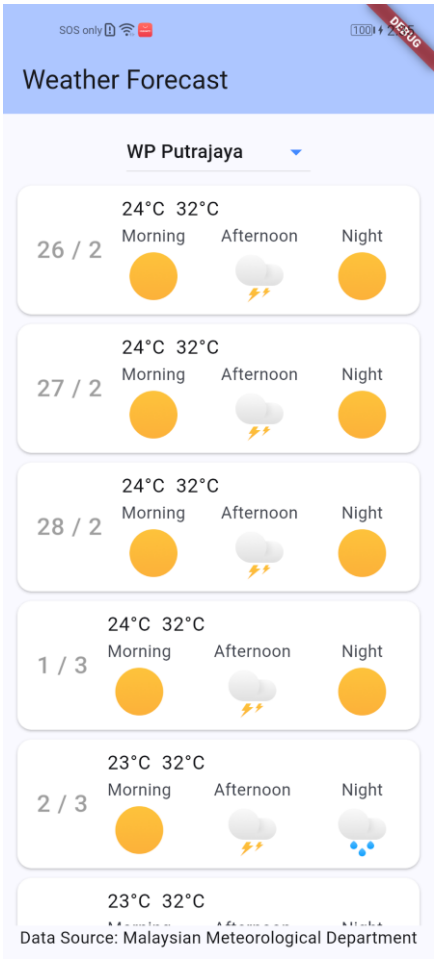
Practical 10: Weather Web API



This project focuses on building a weather application utilizing data sourced from the Malaysian Meteorological Department (MetMalaysia). The application will primarily display weather-related information. Data will be fetched via API calls, requiring parsing of JSON.

1. Create a new **Application** Flutter project named **weather**.

2. Add the **http** and **intl** packages as a dependency using the **Terminal**.

3. In the main.dart file, import the http package.

```
import 'package:http/http.dart' as http;
```

4. For Android deployment, insert the following to the **Manifest file**.

```
<uses-permission android:name="android.permission.INTERNET" />
```

5. The URL for accessing the weather Open Data API is https://developer.data.gov.my/realtime-api/weather

   Click the link above to find out more about the API.

6. Making call to the API, which retrieve 7-day weather for all the states in Malaysia:
   https://api.data.gov.my/weather/forecast?contains=St@location__location_id

7. The data format retrieved from the web server will be in JSON format as follows: -


```
[
    {
        "location": {
            "location_id": "St001",
            "location_name": "Perlis"
        },
        "date": "2025-02-16",
        "morning_forecast": "Tiada hujan",
        "afternoon_forecast": "Tiada hujan",
        "night_forecast": "Tiada hujan",
        "summary_forecast": "Tiada hujan",
        "summary_when": "Sepanjang Hari",
        "min_temp": 24,
        "max_temp": 33
    },
    {
        "location": {
            "location_id": "St001",
            "location_name": "Perlis"
        },
        "date": "2025-02-15",
        "morning_forecast": "Tiada hujan",
        "afternoon_forecast": "Tiada hujan",
    . . .
```


8. Create a new dart file named **forecast.dart**. This class will be used to hold weather records obtained from the API call.

```dart
class Forecast {
  final String date;
  final String morning_forecast;
  final String afternoon_forecast;
  final String night_forecast;
  final String summary_forecast;
  final String summary_when;
  final int min_temp;
  final int max_temp;

  const Forecast({
    required this.date,
    required this.morning_forecast,
    required this.afternoon_forecast,
    required this.night_forecast,
    required this.summary_forecast,
    required this.summary_when,
    required this.min_temp,
    required this.max_temp
  });
```

```
  factory Forecast.fromJson(Map<String, dynamic> json) {
    return switch (json) {
      {
      'date': String date,
      'morning_forecast': String morning_forecast,
      'afternoon_forecast': String afternoon_forecast,
      'night_forecast': String night_forecast,
      'summary_forecast': String summary_forecast,
      'summary_when': String summary_when,
      'min_temp': int min_temp,
      'max_temp': int max_temp
      } =>
          Forecast(
              date: date,
              morning_forecast: morning_forecast,
              afternoon_forecast: afternoon_forecast,
              night_forecast: night_forecast,
              summary_forecast: summary_forecast,
              summary_when: summary_when,
              min_temp: min_temp,
              max_temp: max_temp
          ),
      _ => throw const FormatException('Failed to load forecast.'),
    };
  }
}
```

9. Insert the following images to the **assets/images** folder.

10. Open the **pubspec.yaml** file. Then scroll to the **flutter:** section. You are to insert a sub-section named **assets:**

```
53  flutter:
54
55    # The following line ensures that the Material Icons font is
56    # included with your application, so that you can use the icons in
57    # the material Icons class.
58    uses-material-design: true
59
60    #TODO 1 : Insert assets
61    assets:
62      - assets/images/
63    # To add assets to your application, add an assets section, like this:
64    # assets:
65    #   - images/a_dot_burr.jpeg
66    #   - images/a_dot_ham.jpeg
```

Below the **assets:** section, insert **– assets/images/**. This ensures the app can bundle all the images in this directory.

11. Open the **main.dart** file. Within the class **_MyHomePageState**, insert the following variables and map:

```
final Map<String, String> states = {
  'St001': 'Perlis',
  'St002': 'Kedah',
  'St003': 'Pulau Pinang',
  'St004': 'Perak',
  'St005': 'Kelantan',
  'St006': 'Terengganu',
  'St007': 'Pahang',
  'St008': 'Selangor',
  'St009': 'WP Kuala Lumpur',
  'St010': 'WP Putrajaya',
  'St011': 'Negeri Sembilan',
  'St012': 'Melaka',
  'St013': 'Johor',
  'St501': 'Sarawak',
  'St502': 'Sabah',
  'St503': 'WP Labuan',
};

String? _selectedState; // Store the selected state
Future<List<Forecast>>? forecastData; // Store the Future
```

Each state has a unique code with a prefix 'St'. We declare two variables named **_selectedState** and **forecastData** to hold state code selected by the user and weather forecast data.

12. Within the same class, write a function to perform network request:

```
Future<List<Forecast>> _fetchForecastData(String locationId) async {
  if (locationId.isEmpty) {
    //Handle empty location ID
    return Future.value([]);
  }

  final url =
Uri.parse('https://api.data.gov.my/weather/forecast?contains=$locationId@location__loca
tion_id&sort=date');

  try {
    final response = await http.get(url);

    if (response.statusCode == 200) {
      // If the server did return a 200 OK response, then parse the JSON.
      final jsonData = jsonDecode(response.body) as List;
      return jsonData.map((json) => Forecast.fromJson(json)).toList();
    } else {
      // If the server did not return a 200 OK response, then throw an exception.
      throw Exception('Failed to load forecast : ${response.statusCode}');
    }
  } catch (e) {
    throw Exception('Error fetching forecast data : $e');
  }
}
```

This function will receive a state code (**locationId**) as an input parameter.

13. Within the **build** function of the class **_MyHomePageState,** wrap the Column widget with a Padding widget as follows:

```
body: Center(
  child: Padding(
    padding: const EdgeInsets.all(8.0),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.start,
      children: <Widget>[
        //TODO: Insert a DropdownButton widget

        DropdownButton<String>(
          hint: Text('Select a State'),
          value: _selectedState, // The currently selected value
          iconEnabledColor: Colors.blueAccent,
          borderRadius: BorderRadius.all(Radius.circular(10.0)),
          elevation: 8,
          items: states.entries.map((entry) {
            return DropdownMenuItem<String>(
              value: entry.key, // Value when selected
              child: Text(entry.value), // Text displayed in the dropdown
            );
          }).toList(),
          onChanged: (String? newValue) {
            if (newValue != null) {
              setState(() {
                _selectedState = newValue; // State code
                forecastData = _fetchForecastData(newValue);
              });
            }
          },
        ),
        //TODO: Insert an Expanded widget
        Expanded(
          child: FutureBuilder<List<Forecast>>(
              future: forecastData,
              builder: (context, snapshot) {
                if (snapshot.connectionState == ConnectionState.waiting &&
                    forecastData != null) {
                  return CircularProgressIndicator(
                    strokeAlign: BorderSide.strokeAlignCenter,
                  );
                } else if (snapshot.hasError) {
                  return Text('Error: ${snapshot.error}');
                } else if (snapshot.hasData &&
                    snapshot.data!.isNotEmpty) {
                  return ForecastList(forecastData: snapshot.data!);
                } else {
                  return Text('');
                }
              }),
        ),
        //TODO: Insert a Text widget
        Text('Data Source: Malaysian Meteorological Department'),
      ],
    ),
```

The **ForecastList** is a custom widget that we will insert later.

14. At module level, below the class _MyHomePageState, insert a map for weather status and image file:

```
final Map<String, String> weather_status = {
  'Berjerebu': 'haze.png',
  'Tiada hujan': 'sunny.png',
  'Hujan': 'rainy.png',
  'Hujan di beberapa tempat': 'rainy.png',
  'Hujan di satu dua tempat': 'rainy.png',
  'Hujan di satu dua tempat di kawasan pantai': 'rainy.png',
  'Hujan di satu dua tempat di kawasan pedalaman': 'rainy.png',
  'Ribut petir': 'thunderstorm.png',
  'Ribut petir di beberapa tempat': 'thunderstorm.png',
  'Ribut petir di beberapa tempat di kawasan pedalaman  Scattered':
  'thunderstorm.png',
  'Ribut petir di satu dua tempat': 'thunderstorm.png',
  'Ribut petir di satu dua tempat di kawasan pantai': 'thunderstorm.png',
  'Ribut petir di satu dua tempat di kawasan pedalaman': 'thunderstorm.png',
};
```

15. Next, insert a new stateless widget named **ForecastList**:

```
class ForecastList extends StatelessWidget {
  const ForecastList({super.key});

  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

16. Declare a variable named forecastData as final and make it a required parameter for the default constructor:

```
class ForecastList extends StatelessWidget {
  const ForecastList2({super.key, required this.forecastData});

  final List<Forecast> forecastData;

  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

17. Within the **build** function, replace the **Placeholder** with a **ListView.builder**.

```
@override
Widget build(BuildContext context) {
  return ListView.builder(
      itemCount: forecastData.length,
      itemBuilder: (context, index){});
}
```

18. Within the **itemBuilder** function, write the following code:

```
return ListView.builder(
      itemCount: forecastData.length,
      itemBuilder: (context, index) {
        final forecast = forecastData[index];
        return Card(
```

```
            color: Colors.white,
            child: ListTile(
              title: Text('${forecast.min_temp}°C  ${forecast.max_temp}°C',),
              leading:
                  Text('${DateFormat('yyyy-MM-dd').parse(forecast.date).day} /
                        ${DateFormat('yyyy-MM-dd').parse(forecast.date).month}',
                      style: TextStyle(color: Colors.grey, fontSize: 20,)),
              subtitle: Row(
                crossAxisAlignment: CrossAxisAlignment.center,
                children: [
                  Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      Text('Morning'),
                      weather_status[forecast.morning_forecast] != null
                          ? Image.asset(
                     'assets/images/${weather_status[forecast.morning_forecast].toString()}',
                                width: 48, height: 48,)
                          : Image.asset('assets/images/unknown.png', width: 48, height: 48,),
                    ],
                  ),
                  Expanded(child: SizedBox(),),
                  Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      Text('Afternoon'),
                      weather_status[forecast.afternoon_forecast] != null
                      ? Image.asset(
                    'assets/images/${weather_status[forecast.afternoon_forecast].toString()}',
                           width: 48, height: 48,)
                      : Image.asset('assets/images/unknown.png', width: 48, height: 48,),
                    ],
                  ),
                  Expanded(child: SizedBox(),),
                  Column(
                    mainAxisAlignment: MainAxisAlignment.center,
                    children: [
                      Text('Night'),
                      weather_status[forecast.night_forecast] != null
                          ? Image.asset(
                     'assets/images/${weather_status[forecast.night_forecast].toString()}',
                              width: 48, height: 48,)
                          : Image.asset('assets/images/unknown.png', width: 48, height: 48,
                            ),
                    ],
                  ),
                ],
              ),
            ),
          );
        });
  }
}
```

19. Run the program and select a state from the given list to see 7-day weather forecast records.

TODO: Enhance the app by saving the selected state name using the Shared Preference.